

From DICOM folders to a working submission pipeline in three days

What we built, why each step exists, what broke on the way, and where the model stands now: a first model that scores **macro AUC 0.836** on the 58 expert-labelled studies.

4,407

training studies, 24,371
MRI series

58

with gold labels (all 12
findings)

4,247

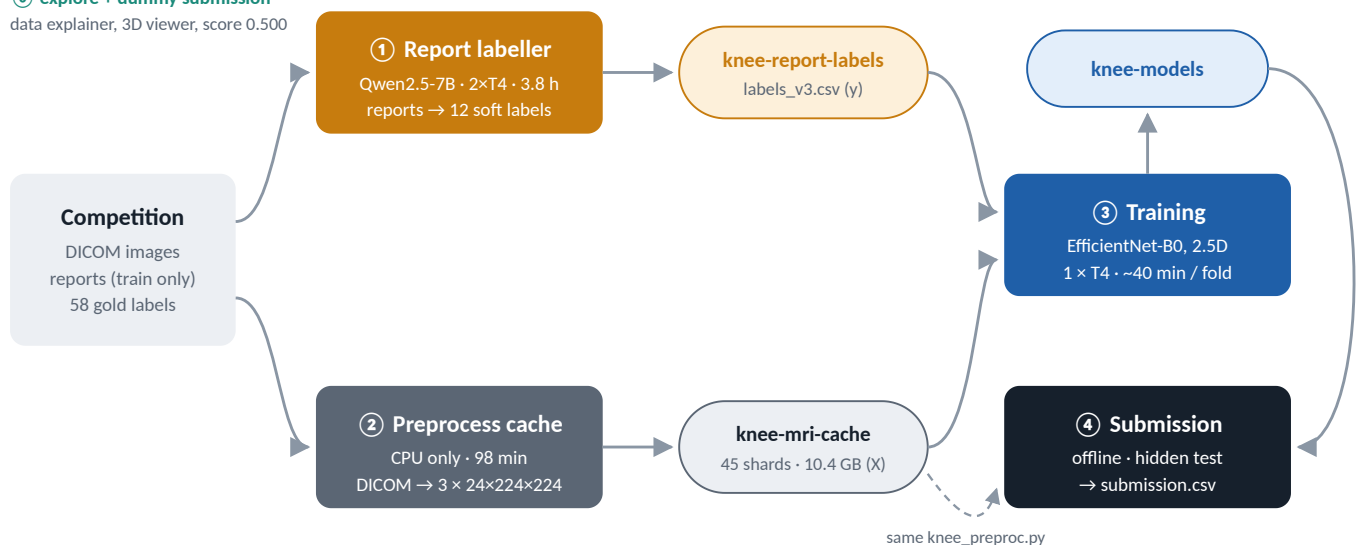
now fully labelled by our
report labeller

0.836

gold macro AUC of the
first model (1 fold)

The pipeline: notebooks and the datasets they produce

🔗 [explore + dummy submission](#)
data explainer, 3D viewer, score 0.500



The core idea in one sentence

Reports exist only for training, so they are turned into **labels**; the model that gets submitted learns from **images only**, because the hidden test set has nothing else.

Scoring: macro ROC-AUC over 12 findings. Only ranking matters, and rare findings count as much as common ones.

Timeline

- ✓ **29 Sep:** data exploration, viewer, dummy submission (0.500)
- ✓ **30 Sep:** labeller v1→v3, full labelling, image cache
- ✓ **1 Oct:** training (after debugging), gold AUC 0.836, submission notebook
- ➔ **Next:** first real LB score, folds 1–4, improvements

Deadlines: entry / team merge **15 Oct** · final submission **22 Oct**

What a knee MRI study looks like

Before any model: learn the file structure and geometry, then prove that a submission survives Kaggle's offline re-run.

Study → Series → Slice

labels: study level · plane/contrast: series level · pixels: slice level

Lessons that shaped everything later

Sort slices by position (`ImagePositionPatient`), never by filename.

Voxels are anisotropic: ~0.3 mm in-plane, ~3-4 mm between slices, hence 2.5D models and several planes per study.

MRI has no absolute units: intensities are normalised per series.

Series are geometrically registered: the interactive 3D viewer shows sagittal, coronal and axial stacks crossing in one patient coordinate system.

Dummy submission: score exactly 0.500, on purpose

Kaggle re-runs a code-competition notebook on ~1,300 hidden studies, offline, within 9 hours. A constant prediction for every study ranks nothing, so AUC = 0.5. What the 0.500 proved is everything around the model:

RISK	HOW THE DUMMY HANDLED IT, AND KEPT IN EVERY LATER NOTEBOOK
crash = no submission at all	a valid <code>submission.csv</code> is written first , before touching images
paths differ in the re-run	data root found automatically; rows and columns taken from <code>sample_submission.csv</code>
compressed DICOMs, no internet	decoders installed offline from a wheels dataset; decoding failures reported per compression type
9-hour limit	decode time measured and extrapolated to the hidden set

Why this order: the dummy took an hour and removed the biggest risk early. Many teams discover offline/codec problems only in the last week.

Turning 4,349 reports into labels

An open-weights LLM (Qwen2.5-7B-Instruct) runs inside Kaggle, so no report text leaves the platform. It grades each finding; the 58 gold studies decide what the grades mean.

Grades, not yes/no

GRADE	DEFAULT	CALIBRATED ON GOLD
present	1	1
minimal	1	Effusion → 0.5 (13 vs 13 in gold) MCL → 0 (1 of 5 positive)
uncertain	0.5	0.5
absent	0	0
not mentioned	0	0

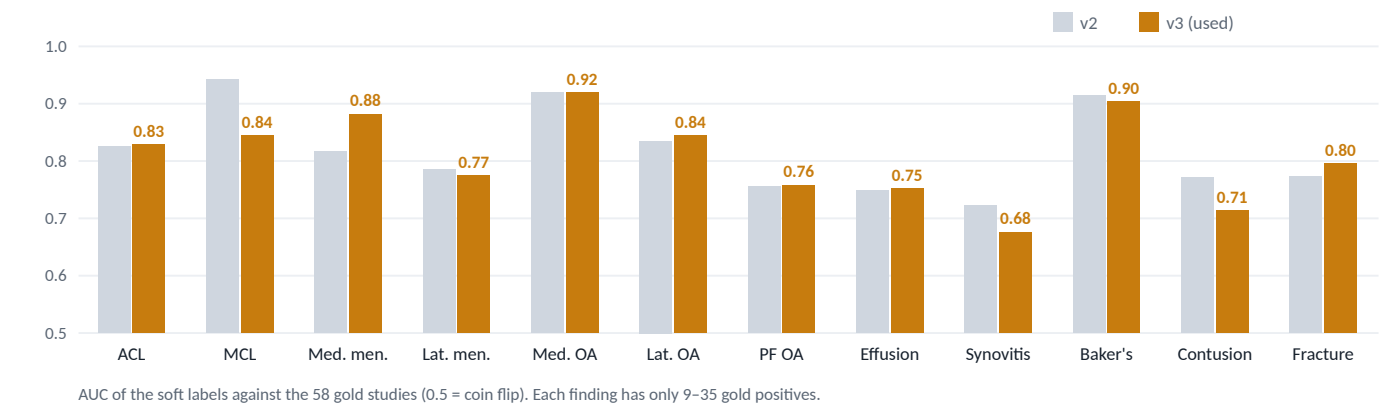
A mapping changes only if ≥ 4 gold studies have that grade, so one odd study can't flip a rule.

Three prompt versions

VER.	WHAT HAPPENED	GOLD AUC
v1	model answered with a bare list → 100 % parse errors	–
v2	answer forced to start with {"language": " "; 15.5 % parse errors	0.818
v3	stricter tear rules, tolerant parser; medial meniscus 0.82 → 0.88	0.809

v2 vs v3 overall is within noise (58 studies). Decision: stop prompt tuning, label everything with v3.

Labeller v3 agreement with gold, per finding



3.8 h

full run, 4,407 reports at 0.32 reports/s on 2×T4 (after fixing an out-of-memory crash with token-budget batching)

3.6 %

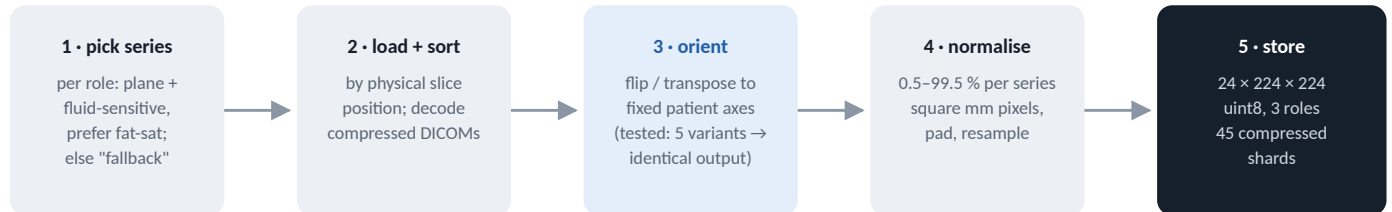
studies with a parse error in some finding; those labels are masked in training, not guessed

ACL • Synovitis

resisted prompt fixes: reports and gold likely disagree (gold probably from image review)

Images, decoded once

Decoding ~100,000 DICOM files every epoch would leave the GPU idle. So every study is converted once into three small, uniform arrays, and the exact code is saved with them.



Three series roles per study

ROLE	BEST FOR
sag_fs sagittal, fluid-sensitive	ACL, menisci, effusion, contusion, Baker's
cor_fs coronal, fluid-sensitive	MCL, meniscal bodies, medial/lateral OA
ax_fs axial, fluid-sensitive	patellofemoral OA, synovitis, Baker's

In the training pool all three roles were available for 100 % of studies.

Design decisions

One shared module, `knee_preproc.py`, saved inside the dataset. Training reads the cache with it; the submission runs the same `process_study()` on hidden test DICOMs, so train and test are prepared identically.

Shards, because Kaggle keeps ≤ 500 output files and ≤ 20 GB: 45 files, 10.4 GB.

Left vs right knee is not guessed; laterality and position are stored for later.

98 min

for 4,407 studies on 4 CPU cores, no GPU hours used

2.45 MB

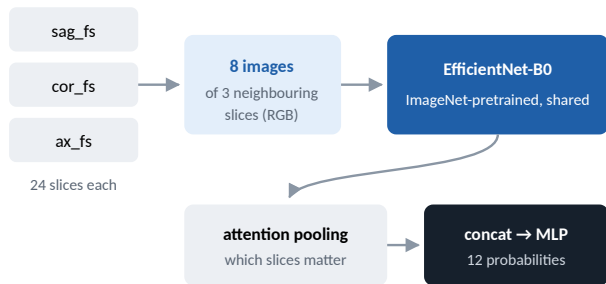
per study, compressed (3 series)

2 ms

to fetch a study from RAM during training (vs 140–170 ms from disk)

The model, the debugging, the first result

2.5D multi-view network



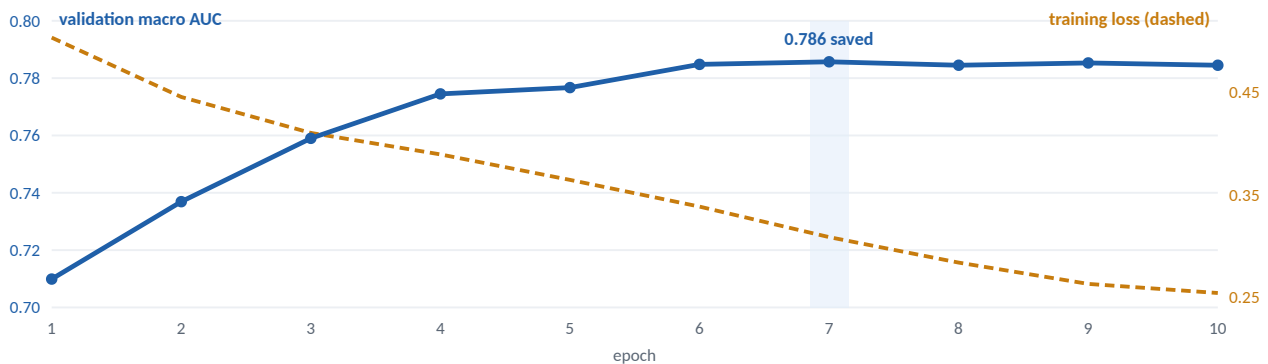
Soft-label BCE with missing labels masked · mirror-the-whole-knee augmentation · 58 gold studies never trained on.

What broke, and the fix

SYMPTOM	CAUSE → FIX
first epoch never finished	reading shards from disk per study → preload 15.9 GB into RAM (0.8 min)
7.5 s/step, kernel died	made visible by step logging, measured with a diagnostic notebook
<code>misaligned address</code>	DataParallel (2 GPUs) broken here → 1 GPU + channels_last
corrupted reads (zlib)	file handle shared across processes → one reader per process

Result: 7.5 → **0.48 s/step**, 3.7 min per epoch, RAM stable at 57 %.

Fold 0: 10 epochs, 40 minutes



Validation = 903 studies from other site groups, scored against report labels. After epoch 7 the training loss keeps falling but validation stops improving: the model starts memorising. The epoch-7 weights are saved.

0.836

gold macro AUC: 58 expert-labelled studies, never trained on. Rough range 0.79–0.88 given the tiny set.

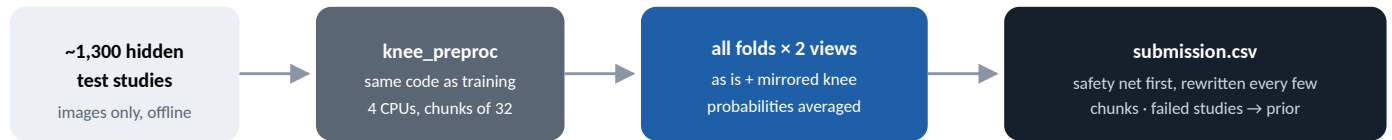
0.786

best validation AUC on report labels (partly measures agreement with the labeller's mistakes)

> 0.81

the image model beats the labeller's own agreement with gold: it can see what reports don't say

From trained weights to a leaderboard score



Inputs: competition data · knee-models · knee-mri-cache (only for `knee_preproc.py`) · knee-dicom-wheels. Estimated hidden-set runtime ≈ 30 min preprocessing + a few minutes of prediction, far below the 9 h limit.

Immediate next steps

- 1 Create **knee-dicom-wheels** and submit with fold 0 → first real leaderboard score
- 2 Train **folds 1-4** (~2.7 h), add to `knee-models`, resubmit as a 5-fold ensemble
- 3 Look at **per-finding AUC**; the weak ones decide where to improve
- 4 Check the **decode-failure rate** on the hidden set (optional diagnostic stop)

Improvement ideas, roughly by expected value

Bigger backbone (EfficientNet-B2, ConvNeXt-Nano): one config line, slower but usually better.

More slices / higher resolution for small structures (ACL, menisci).

Use both GPUs by training two folds in parallel processes (not `DataParallel`).

Per-finding loss weights or gold-informed fine-tuning for the weakest findings.

Lessons from the three days

Measure before fixing. Two blind fixes for the slow training missed; one 5-minute diagnostic found `DataParallel` immediately.

Make failures visible. Step-level progress, decode reports and "safety net first" turned silent hangs into readable logs.

Stop polishing at the noise floor. With 58 gold studies, prompt tweaks beyond v3 would have chased noise; labelling everything was worth more.